Practical 6: K-Means and Hierarchical Clustering on Sales Data

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram
```

In [2]:
```
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
df.head()
```

Out[2]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	ADDRESSLINE1	ADDRESSLINE2	CITY	STATE	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	2	2003	...	897 Long Airport Avenue	NaN	NYC	NY	10022	USA	NaN	Yu	Kwai	Small
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	5	2003	...	59 rue de l'Abbaye	NaN	Reims	NaN	51100	France	EMEA	Henriot	Paul	Small
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	7	2003	...	27 rue du Colonel Pierre Avia	NaN	Paris	NaN	75508	France	EMEA	Da Cunha	Daniel	Medium
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	8	2003	...	78934 Hillside Dr.	NaN	Pasadena	CA	90003	USA	NaN	Young	Julie	Medium
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	10	2003	...	7734 Strong St.	NaN	San Francisco	CA	NaN	USA	NaN	Brown	Julie	Medium

5 rows × 25 columns

In [3]:
```
df.shape
```

Out[3]:
```
(2823, 25)
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   ORDERNUMBER       2823 non-null   int64  
 1   QUANTITYORDERED   2823 non-null   int64  
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64  
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   str    
 6   STATUS            2823 non-null   str    
 7   QTR_ID            2823 non-null   int64  
 8   MONTH_ID          2823 non-null   int64  
 9   YEAR_ID           2823 non-null   int64  
 10  PRODUCTLINE       2823 non-null   str    
 11  MSRP              2823 non-null   int64  
 12  PRODUCTCODE       2823 non-null   str    
 13  CUSTOMERNAME      2823 non-null   str    
 14  PHONE             2823 non-null   str    
 15  ADDRESSLINE1      2823 non-null   str    
 16  ADDRESSLINE2      302 non-null    str    
 17  CITY              2823 non-null   str    
 18  STATE             1337 non-null   str    
 19  POSTALCODE        2747 non-null   str    
 20  COUNTRY           2823 non-null   str    
 21  TERRITORY         1749 non-null   str    
 22  CONTACTLASTNAME   2823 non-null   str    
 23  CONTACTFIRSTNAME  2823 non-null   str    
 24  DEALSIZE          2823 non-null   str    
dtypes: float64(2), int64(7), str(16)
memory usage: 551.5 KB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
ORDERNUMBER           0
QUANTITYORDERED       0
PRICEEACH             0
ORDERLINENUMBER       0
SALES                 0
ORDERDATE             0
STATUS                0
QTR_ID                0
MONTH_ID              0
YEAR_ID               0
PRODUCTLINE           0
MSRP                  0
PRODUCTCODE           0
CUSTOMERNAME          0
PHONE                 0
ADDRESSLINE1          0
ADDRESSLINE2       2521
CITY                  0
STATE              1486
POSTALCODE           76
COUNTRY               0
TERRITORY          1074
CONTACTLASTNAME       0
CONTACTFIRSTNAME      0
DEALSIZE              0
dtype: int64
```

In [6]:
```
features = ["QUANTITYORDERED", "PRICEEACH", "SALES", "MSRP"]
X = df[features]
X.describe()
```

Out[6]:

	QUANTITYORDERED	PRICEEACH	SALES	MSRP
count	2823.000000	2823.000000	2823.000000	2823.000000
mean	35.092809	83.658544	3553.889072	100.715551
std	9.741443	20.174277	1841.865106	40.187912
min	6.000000	26.880000	482.130000	33.000000
25%	27.000000	68.860000	2203.430000	68.000000
50%	35.000000	95.700000	3184.800000	99.000000
75%	43.000000	100.000000	4508.000000	124.000000
max	97.000000	100.000000	14082.800000	214.000000

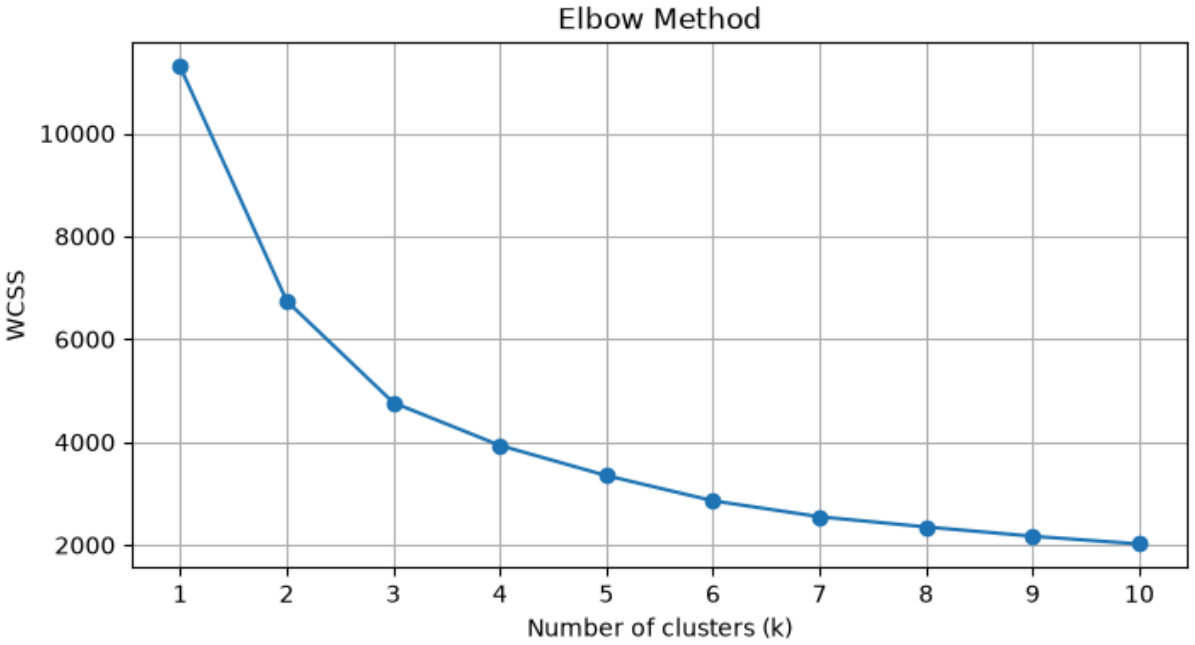
In [7]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [8]:
```
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
    print(f"k = {k:2d}   WCSS = {kmeans.inertia_:.2f}")

plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```

```
k =  1   WCSS = 11292.00
k =  2   WCSS = 6741.69
k =  3   WCSS = 4766.01
k =  4   WCSS = 3934.89
k =  5   WCSS = 3348.27
k =  6   WCSS = 2858.82
```

```
k =  7   WCSS = 2548.54
k =  8   WCSS = 2350.66
k =  9   WCSS = 2169.97
k = 10   WCSS = 2019.75
```



In [9]:
```
k = 3
kmeans = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)
df["Cluster"].value_counts().sort_index()
```

Out[9]:
```
Cluster
0    975
1    852
2    996
Name: count, dtype: int64
```

In [10]:
```
pd.DataFrame(scaler.inverse_transform(kmeans.cluster_centers_), columns=features).round(2)
```

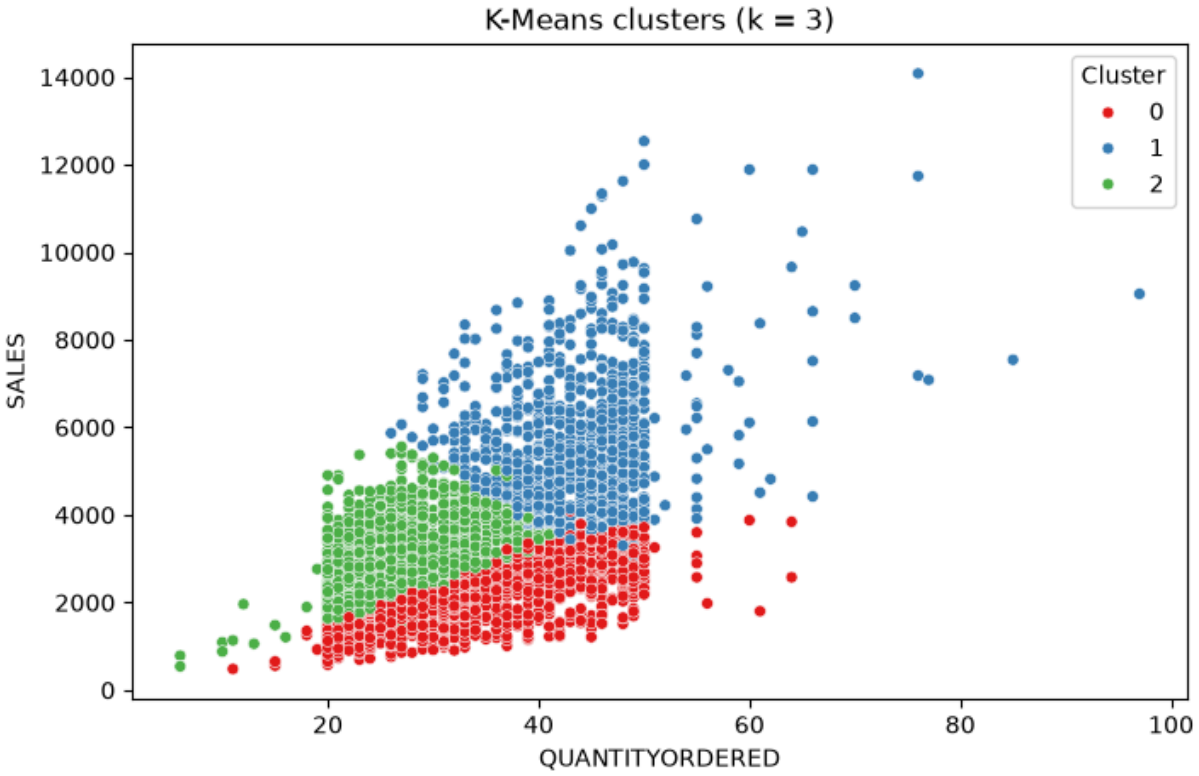
Out[10]:

	QUANTITYORDERED	PRICEEACH	SALES	MSRP
0	35.21	59.55	2101.08	64.15
1	43.73	97.49	5722.35	127.95
2	27.61	95.43	3123.73	113.23

In [11]:
```
print("Silhouette score:", round(silhouette_score(X_scaled, df["Cluster"]), 4))
```

```
Silhouette score: 0.3509
```

In [12]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x="QUANTITYORDERED", y="SALES", hue="Cluster", palette="Set1",
s=25)
plt.title(f"K-Means clusters (k = {k})")
plt.show()
```

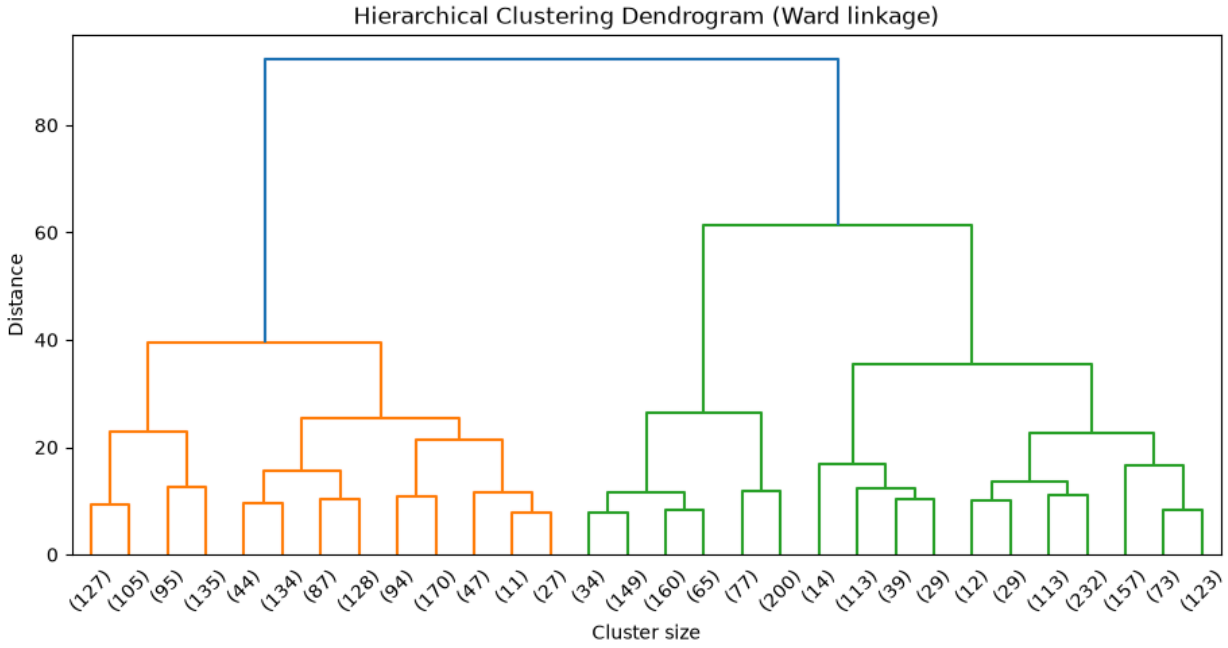


In [13]:
```
pd.crosstab(df["Cluster"], df["DEALSIZE"])
```

Out[13]:

DEALSIZE	Large	Medium	Small
Cluster			
0	0	144	831
1	157	691	4
2	0	549	447

In [14]:
```
Z = linkage(X_scaled, method="ward")
plt.figure(figsize=(11, 5))
dendrogram(Z, truncate_mode="lastp", p=30)
plt.title("Hierarchical Clustering Dendrogram (Ward linkage)")
plt.xlabel("Cluster size")
plt.ylabel("Distance")
plt.show()
```



In [15]:
```
hc = AgglomerativeClustering(n_clusters=k, linkage="ward")
df["HC_Cluster"] = hc.fit_predict(X_scaled)
df["HC_Cluster"].value_counts().sort_index()
```

Out[15]:
```
HC_Cluster
0    1204
1     934
2     685
Name: count, dtype: int64
```

In [16]:
```
print("Silhouette score (hierarchical):", round(silhouette_score(X_scaled,
df["HC_Cluster"]), 4))
```

```
Silhouette score (hierarchical): 0.3153
```